TOOLKIT · BACKEND ENGINEERING HANDBOOK

NGINX COMPLETE HANDBOOK

Reverse Proxies, HTTP Routing, Production Load Balancing, TLS Termination & Incident Mastery

Phase	II — Backend Infrastructure
Tool ID	09 of 13
Curriculum Scope	Complete 2-Chunk Comprehensive Curriculum
Deliverable Type	Consolidated Tool Engineering Handbook (Release Build)
Source Authority	content/nginx/ (01 and 02 markdown sources)
Target Audience	Backend Software Engineers, SDE-1 / SDE-2, Systems Engineers
Engineering Focus	Reverse Proxying, Upstream Routing, TLS Offloading, Load Balancing & Production Triage (502/504)

This comprehensive handbook provides an authoritative, production-grade guide to **Nginx** for backend engineering. Spanning three rigorous pedagogical layers—Mental Models, Practical Engineering, and Production Incident Mastery—it equips engineers to design resilient traffic management layers, master server blocks, location matching algorithms, and proxy_pass path semantics, configure production-grade upstream load balancing (Round Robin, Least Connections, IP Hash), terminate TLS certificates securely, tune timeouts and connection keepalive, enforce rate limits and request size boundaries, and systematically diagnose 502 Bad Gateway, 504 Gateway Timeout, and upstream routing regressions.

Curriculum Structure & Core Topics

Part I — Reverse Proxies, HTTP Routing & Core Mental Models: Reverse proxy vs forward proxy vs load balancer, event-driven master/worker architecture, configuration hierarchy, server blocks, location matching rules (=, ^~, ~, prefix), proxy_pass URI mapping and trailing-slash traps, forwarded headers, and 502/504 triage.

Part II — Production Load Balancing, TLS, Reliability, Security & Incident Mastery: Upstream blocks, load balancing strategies (round robin, least_conn, ip_hash), passive vs active health checks, connection reuse (upstream keepalive), TLS termination, HTTP to HTTPS redirection, rate limiting, access control, and incident runbooks.

PART I: REVERSE PROXIES, HTTP ROUTING & CORE MENTAL MODELS

Chunk 1 of 2 · Backend Engineering Handbook

Nginx — Chunk 1 of 2: Reverse Proxies, HTTP Routing & Core Mental Models

Backend Engineering Tools & Infrastructure Handbook

Source file: `content/nginx/01-nginx-reverse-proxies-routing-core-mental-models.md`

Learning Objectives

By the end of this chapter, you should be able to:

- Explain what Nginx does and why backend systems use it.
- Distinguish a reverse proxy from a forward proxy and a load balancer.
- Trace an HTTP request from a client through Nginx to a Spring Boot application.
- Understand Nginx configuration structure, server blocks, and location matching.
- Configure Nginx to serve static files and reverse-proxy API requests.
- Forward the correct client and protocol headers to your backend.
- Diagnose common `502`, `504`, `403`, and routing failures.
- Explain Nginx's role in a production backend architecture.

This chapter focuses on fundamentals and practical workflows. Chunk 2 will deepen production operations, load balancing, TLS, reliability, security, performance, and incident mastery.

Layer 1 — Fundamentals & Mental Models

1. What Is Nginx?

Nginx (pronounced *engine-x*) is a web server and reverse proxy commonly used in backend infrastructure.

It can:

- Serve static assets such as HTML, CSS, JavaScript, images, and downloads.
- Forward HTTP requests to backend applications.
- Distribute requests across multiple backend instances.
- Terminate TLS connections.
- Apply routing rules, timeouts, and request-size limits.
- Support caching and other traffic-management features.

Imagine your Spring Boot API runs on port `8080`. Instead of exposing that application port directly to every client, you place Nginx in front of it.

```
Client → Nginx → Spring Boot API → PostgreSQL
```

Nginx becomes the public-facing entry point. Your application can concentrate on business logic while Nginx handles HTTP traffic management.

Why not expose Spring Boot directly?

You can expose Spring Boot directly, and small applications sometimes do. Nginx becomes useful when you need a dedicated traffic layer.

Concern	Spring Boot application	Nginx
Business logic	Primary responsibility	Not its role
Database operations	Yes	No
API validation	Yes	Limited request-level controls
Static file serving	Possible	Efficient and common
Reverse proxying	Usually not its main role	Core capability
TLS termination	Supported	Common deployment pattern
Traffic distribution	Possible through other components	Built-in load-balancing capabilities

Mental model: Nginx manages the traffic path; your backend manages the application behavior.

2. Forward Proxy vs Reverse Proxy

These two proxies sit on opposite sides of a communication.

```
Forward proxy:
Client → Forward Proxy → Internet

Reverse proxy:
Internet Client → Reverse Proxy → Backend Server
```

Forward proxy

A forward proxy represents or mediates traffic from clients.

For example, a company may route employee web traffic through a proxy to enforce access policies or filter destinations.

The client typically knows it is using the proxy.

Reverse proxy

A reverse proxy represents one or more backend servers.

A client requests:

`https://api.example.com/orders/123`

Nginx receives the request and forwards it to an internal application such as:

`http://127.0.0.1:8080/orders/123`

The client does not need to know the internal address or port.

Why reverse proxies matter

1. **Hide internal topology:** clients don't need direct access to application instances.
2. **Centralize traffic handling:** routing and TLS configuration can live in one layer.
3. **Scale horizontally:** requests can be distributed across several backend instances.
4. **Separate concerns:** traffic management can change without rewriting business logic.
5. **Provide a consistent entry point:** multiple services can be exposed through one domain.

A reverse proxy is not automatically a firewall or a complete security boundary. Network rules and application-level security still matter.

3. Reverse Proxy vs Load Balancer

A reverse proxy receives requests and forwards them to backend servers. A load balancer distributes requests among multiple eligible backends.

These are not mutually exclusive categories: **Nginx can act as both a reverse proxy and a load balancer.**

Single backend:

```
Client → Nginx → Spring Boot :8080
```

Multiple backends:

```
                    ■→ Spring Boot :8081
Client → Nginx ■■■■■■■→ Spring Boot :8082
                    ■→ Spring Boot :8083
```

With multiple backends, Nginx can use policies such as round-robin, weighted distribution, or least connections, depending on the configuration and edition.

We will configure and analyze load balancing in Chunk 2.

4. The HTTP Request Lifecycle

Consider a request to retrieve an order:

`GET https://api.example.com/orders/123`

A typical request path looks like this:

```
1. Client resolves api.example.com through DNS.
2. Client establishes a TCP connection to the public endpoint.
3. TLS is negotiated if HTTPS is used.
4. Nginx receives and parses the HTTP request.
5. Nginx selects a server block and location.
6. Nginx forwards the request to the configured upstream.
7. Spring Boot processes the request.
8. Spring Boot returns an HTTP response.
9. Nginx relays the response to the client.
```

The exact path depends on the deployment. For example, TLS may terminate at a cloud load balancer before the request reaches Nginx.

What can go wrong at each stage?

Stage	Example failure
DNS	Domain resolves to the wrong IP
TCP connection	Connection refused or timeout
TLS	Certificate or handshake failure
Nginx routing	Wrong server block or location
Upstream connection	Spring Boot is down; often a `502`
Upstream response time	Backend exceeds proxy timeout; often a `504`
Application	Spring Boot returns `400`, `404`, or `500`
Response path	Connection drops or response is interrupted

A useful debugging rule is to identify **which layer generated the failure** rather than assuming every HTTP error originates in the application.

5. Nginx Architecture: Master and Worker Processes

Nginx typically uses a master process and worker processes.

- **Master process:** reads configuration, manages workers, and coordinates reloads.
- **Worker processes:** handle network connections and process requests.
- **Configuration files:** define listeners, routing rules, upstreams, and other behavior.
- **Logs:** provide evidence about incoming requests and failures.

Nginx is designed around event-driven, non-blocking connection handling. A worker can manage many concurrent connections without requiring one operating-system thread per connection.

This is especially useful for high numbers of mostly I/O-bound network connections.

However, event-driven networking does not make slow upstream applications magically faster. If a backend is overloaded, proxying more requests can still increase latency and resource pressure.

Important distinction

Nginx's worker model and your application's concurrency model are separate. A large number of client connections handled by Nginx does not imply that the Spring Boot application can process the same number of expensive

operations without saturation.

6. Configuration File Structure

A common Linux installation uses:

- `/etc/nginx/nginx.conf` — main configuration.
- `/etc/nginx/conf.d/` — commonly used for additional configuration files.
- `/etc/nginx/sites-available/` and `/etc/nginx/sites-enabled/` — commonly used by Debian/Ubuntu packaging.
- `/var/log/nginx/access.log` — access log.
- `/var/log/nginx/error.log` — error log.

Exact paths and included files vary by operating system and installation method. Inspect the active configuration rather than assuming a directory is enabled.

Main configuration concepts

Directive or block	Purpose
`worker_processes`	Controls worker process count
`events`	Configures connection-processing behavior
`http`	Contains HTTP server and upstream configuration
`server`	Defines a virtual server, commonly selected by port and hostname
`listen`	Defines an address or port to listen on
`server_name`	Identifies hostnames handled by a server block
`location`	Matches request URIs
`proxy_pass`	Forwards requests to an upstream
`root`	Defines a filesystem root for static files
`index`	Defines default index filenames
`include`	Loads additional configuration files

A minimal configuration

```
events {
    worker_connections 1024;
}

http {
    server {
        listen 8088;
        server_name localhost;

        location / {
            return 200 "Nginx is running\n";
        }
    }
}
```

This example demonstrates the hierarchy and a simple response. It is not a production deployment configuration.

Test before applying changes

```
nginx -t
```

If Nginx is installed outside the standard configuration path, specify the appropriate configuration file:

```
nginx -t -c /path/to/nginx.conf
```

A successful configuration test means Nginx can parse the configuration and perform relevant checks. It does not prove that every upstream is reachable or that every route behaves correctly.

7. Server Blocks and Host-Based Routing

A server block defines how Nginx handles requests matching its listening address, port, and host-selection rules.

Example:

```
server {
    listen 80;
    server_name api.example.com;

    location / {
        return 200 "API entry point\n";
    }
}

server {
    listen 80;
    server_name admin.example.com;

    location / {
        return 200 "Admin entry point\n";
    }
}
```

Both server blocks listen on port `80`, but their `server_name` values allow Nginx to route requests for different hostnames to different configurations.

This is called **name-based virtual hosting**.

Common mistakes

- The requested hostname does not match the intended `server_name`.
- Another server block becomes the default for the listening address.
- The configuration file exists but is not included by the active configuration.
- A change was made, but Nginx was never reloaded.
- DNS points to a different machine than expected.

Inspect the complete active configuration with:

```
nginx -T
```

Use care when sharing this output: configuration dumps can contain internal hostnames, paths, and sensitive operational details.

8. Location Matching: How Nginx Chooses a Route

The `location` directive matches the request URI path.

Common forms include:

```
location = /health {
    return 200 "healthy\n";
}

location /api/ {
    # API routes
}

location ^~ /assets/ {
    # Static assets
}

location ~ \.php$ {
    # Example of a case-sensitive regular-expression match
}
```

The essential matching rules

- `location = /path` — exact match; if it matches, it wins immediately.
- `location ^~ /prefix/` — prefix match that prevents a later regular-expression match from taking precedence.
- `location ~ pattern` — case-sensitive regular expression.
- `location ~* pattern` — case-insensitive regular expression.
- `location /prefix/` — ordinary prefix match.

For ordinary prefix locations, Nginx generally remembers the longest matching prefix and then checks regular-expression locations, unless the chosen prefix uses `^~`. Regular expressions are evaluated in configuration order, and the first matching regular expression wins.

The full algorithm has nuances, so don't rely on the simplistic claim that Nginx always chooses the longest string in the file.

Example

```
server {
    listen 8088;
    server_name localhost;

    location = /health {
        return 200 "exact health match\n";
    }

    location /api/ {
        return 200 "API prefix match\n";
    }

    location / {
        return 200 "fallback match\n";
    }
}
```

Test each route:

```
curl -i http://localhost:8088/health
curl -i http://localhost:8088/api/orders
curl -i http://localhost:8088/about
```

Expected behavior:

- `/health` matches the exact location.
- `/api/orders` matches the `/api/` prefix.
- `/about` falls back to `/`.

Trailing slash traps

`/api` and `/api/` are different URI paths. A location declared as `location /api/` does not directly express an exact match for `/api`.

If you want `/api` to redirect to `/api/`, you can define the behavior explicitly:

```
location = /api {
    return 301 /api/;
}

location /api/ {
    # API handling
}
```

Use redirects deliberately. A redirect changes the client-visible URL and can affect HTTP methods, caching, and API consumers.

9. `proxy_pass`: Forwarding Requests to Spring Boot

This is one of the most important Nginx directives for backend engineers.

Assume Spring Boot listens on:

`http://127.0.0.1:8080`

You want Nginx to receive requests on port `80` and forward API traffic to the application.

```
server {
    listen 80;
    server_name api.example.com;

    location /api/ {
        proxy_pass http://127.0.0.1:8080;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

With this configuration, a request for:

`http://api.example.com/api/orders/123`

is forwarded upstream with the URI path preserved as:

`/api/orders/123`

This matters because your Spring Boot controller must expect the path that actually reaches the application.

Why forward these headers?

Header	Why it matters
`Host`	Preserves the original host information
`X-Real-IP`	Passes a client-address value
`X-Forwarded-For`	Carries a chain of proxy/client addresses
`X-Forwarded-Proto`	Indicates the original request scheme, such as HTTP or HTTPS

These headers are only trustworthy if your proxy chain is configured correctly. A backend directly exposed to untrusted clients must not blindly trust client-supplied forwarding headers. Configure trusted proxies and the framework's forwarded-header behavior deliberately.

Important: `proxy_pass` trailing-slash behavior

These configurations can forward different URI paths.

Configuration A — no URI component:

```
location /api/ {
    proxy_pass http://127.0.0.1:8080;
}
```

A request to `/api/orders` is normally forwarded as `/api/orders`.

Configuration B — URI component `/`:

```
location /api/ {
    proxy_pass http://127.0.0.1:8080/;
}
```

For a straightforward prefix location, the matched `/api/` prefix is replaced by `/`, so `/api/orders` is forwarded as `/orders`.

This is a frequent source of backend `404` errors. If Spring Boot expects `/api/orders`, removing `/api/` upstream may break the route. Conversely, if the application expects `/orders`, stripping the prefix may be intentional.

Rule: Test the actual upstream URI. Never add or remove a trailing slash based on appearance alone.

10. Serving Static Files

Nginx can serve files directly without involving Spring Boot.

Example directory:

```
/var/www/myapp/
■■■ index.html
■■■ assets/
■   ■■■ app.js
■   ■■■ app.css
■■■ images/
    ■■■ logo.png
```

Configuration:

```
server {
    listen 8088;
    server_name localhost;

    root /var/www/myapp;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

`try_files` checks candidate filesystem paths and returns `404` if none exists in this example.

`root` VS `alias`

These directives are related but not identical.

- `root` appends the request URI to the configured root path.
- `alias` replaces the matched location prefix with a specified filesystem path.

Example using `alias`:

```
location /downloads/ {
    alias /srv/app-downloads/;
}
```

A request for `/downloads/report.pdf` maps to `/srv/app-downloads/report.pdf`.

Be careful with path mapping and trailing slashes. Incorrect filesystem mapping can cause missing-file errors or unintentionally expose files.

11. Local Lab: Put Nginx in Front of a Backend

You can perform this lab on your development machine. The examples assume Nginx and a backend are reachable from the same host. Containerized deployments need appropriate service names and networking instead of blindly using `127.0.0.1`.

Step 1 — Start a test backend

If you already have a Spring Boot API listening on port `8080`, use it. Otherwise, create a temporary HTTP service for testing:

```
python3 -m http.server 8080
```

This is a simple static-file server, not a Spring Boot API. Keep it running in one terminal.

Verify the backend:

```
curl -i http://127.0.0.1:8080/
```

Step 2 — Configure Nginx

Create a server block in a configuration file included by your Nginx installation:

```
server {
    listen 8088;
    server_name localhost;

    location / {
        proxy_pass http://127.0.0.1:8080;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Ensure the file is included by the active `http` configuration. Avoid overwriting the main configuration blindly.

Step 3 — Validate and reload

```
nginx -t
```

After a successful test, reload using the service manager or installation-specific command. For a typical Linux systemd installation:

```
sudo systemctl reload nginx
```

Check service status:

```
sudo systemctl status nginx
```

On macOS or a containerized setup, the service commands and paths may differ.

Step 4 — Test through Nginx

```
curl -i http://127.0.0.1:8088/
```

Compare the response from port `8088` with the direct backend response from port `8080`.

Step 5 — Inspect logs

```
sudo tail -f /var/log/nginx/access.log
```

In another terminal:

```
sudo tail -f /var/log/nginx/error.log
```

Generate a request and observe which log entries appear. Your installation may use different paths or permissions.

Step 6 — Deliberately create a failure

Stop the test backend, then run:

```
curl -i http://127.0.0.1:8088/
```

Nginx will generally return `502 Bad Gateway` when it cannot connect to the configured upstream. Inspect the error log, restart the backend, and repeat the request.

Lab result: You have demonstrated the difference between the public-facing proxy and the upstream application, plus the role of logs in diagnosing an upstream failure.

12. Common Nginx Errors and First Checks

Symptom	Likely areas to inspect	First action
`502 Bad Gateway`	Backend down, wrong upstream host/port, refused connection	`curl` the upstream directly
`504 Gateway Timeout`	Upstream too slow or unreachable, network delay	Check backend latency and Nginx error log
`403 Forbidden`	Filesystem permissions, access rules, directory traversal rules	Inspect the path and Nginx worker permissions
`404 Not Found`	Wrong location, URI rewrite, missing file, backend route	Compare requested and upstream paths
`413 Content Too Large`	Request-body size limit	Inspect `client_max_body_size` and application requirements

Symptom	Likely areas to inspect	First action
`400 Bad Request`	Malformed request, host or protocol mismatch, application validation	Determine which layer generated the response
`504` only under load	Saturated backend, slow dependency, too-short timeout	Measure latency and resource saturation before changing timeouts

These are diagnostic starting points, not definitive mappings. An application can itself return a `502` or `504`, and a proxy can return a `404` based on its own configuration.

13. Security Fundamentals

Even a basic reverse proxy configuration needs sound security practices.

1. **Don't expose unnecessary ports.** Clients should reach the intended public entry point; internal backend ports should be restricted by network controls.
2. **Protect secrets.** Don't commit credentials, private keys, or sensitive environment values to Git.
3. **Validate forwarding headers.** Trust them only from known proxy infrastructure.
4. **Limit request sizes appropriately.** An oversized upload should not consume unlimited resources.
5. **Keep software patched.** Use a supported Nginx release and maintain the underlying operating system or container image.
6. **Protect logs.** Access logs may contain identifiers, query strings, or other sensitive data.
7. **Use TLS for public production traffic.** TLS termination and certificate operations are covered in Chunk 2.
8. **Avoid reflexive timeout increases.** A longer timeout can keep resources occupied while an unhealthy backend remains slow.

Nginx is one layer of defense, not a replacement for authentication, authorization, input validation, safe database access, or application security.

Layer 2 — Practical Engineering & Workflows

14. The Backend Engineer's Nginx Debugging Workflow

When an API endpoint fails, proceed from the application outward rather than changing several settings at once.

Step A — Test the application directly

```
curl -i http://127.0.0.1:8080/actuator/health
```

Use a real endpoint appropriate for your application. The Spring Boot Actuator health endpoint is available only if Actuator is installed and the endpoint is enabled and accessible.

If this fails, investigate the application, port, and dependencies before blaming Nginx.

Step B — Verify Nginx configuration

```
sudo nginx -t
sudo nginx -T
```

Check server names, listeners, included files, locations, upstream addresses, and `proxy_pass` paths.

Step C — Verify the listening ports

On Linux:

```
sudo ss -ltnp
```

Confirm that Nginx and the backend are listening on the intended addresses and ports.

Step D — Inspect logs

```
sudo tail -n 100 /var/log/nginx/error.log
sudo tail -n 100 /var/log/nginx/access.log
```

Look for the timestamp of the failing request, the upstream address, connection errors, and response status.

Step E — Reproduce through the proxy

```
curl -i \
  -H 'Host: api.example.com' \
  http://127.0.0.1/api/orders
```

This tests the selected host-based routing rule on a local Nginx listener. Adapt the port and path to your environment.

Step F — Change one thing and retest

After making a targeted change:

```
sudo nginx -t && sudo systemctl reload nginx
```

This is a common Linux systemd workflow. It does not apply unchanged to every installation.

Re-run the same request and compare the result. A successful reload is not proof that the application behavior is correct.

15. Backend Deployment Architecture

A common architecture is:

```
Client
  |
  | HTTPS
  v
Nginx
  |
  | HTTP over a restricted internal network
  v
Spring Boot API
  |
  +------> PostgreSQL
  |
  +------> Redis
  |
  +------> Kafka
```

The components have distinct responsibilities:

- **Nginx:** public HTTP entry point, routing, and potentially TLS termination.
- **Spring Boot:** API behavior and business rules.
- **PostgreSQL:** durable relational data.
- **Redis:** caching or other suitable low-latency data operations.
- **Kafka:** asynchronous event streaming when the architecture requires it.

Nginx does not replace the database, message broker, or application. It provides a traffic-management layer in front of services.

In a Docker Compose environment, `127.0.0.1` inside the Nginx container refers to that Nginx container itself. To reach a backend in another Compose service, use the service name on the shared network, such as `http://api:8080`, provided the service is actually named `api`.

This localhost distinction is one of the most common deployment mistakes.

Layer 3 — Production Engineering & Interview Mastery

16. Interview Questions and Answers

Q1. What is Nginx?

Nginx is a web server and reverse proxy that can serve static content, route requests, forward traffic to backends, and distribute requests across upstream servers.

Q2. What is the difference between a forward proxy and a reverse proxy?

A forward proxy mediates client traffic to destinations. A reverse proxy receives client requests on behalf of backend servers and forwards them to those servers.

Q3. Is Nginx a load balancer?

It can be. Nginx supports reverse proxying to one backend or distributing requests across multiple upstream servers.

Q4. What happens when Nginx returns `502 Bad Gateway`?

It often means Nginx could not obtain a valid response from the upstream. Check backend health, connection refusal, address/port correctness, networking, and the error log.

Q5. How does `502` differ from `504`?

A `502` commonly indicates an invalid or unavailable upstream response. A `504` indicates that a gateway or proxy did not receive a timely upstream response. The exact cause must be established from logs and request behavior.

Q6. What does `proxy_pass` do?

It forwards a request to a configured upstream. Its URI component and interaction with the matched location affect the path that reaches the backend.

Q7. Why set `X-Forwarded-For`?

It can carry client and proxy address information through the proxy chain. It must be interpreted using a trusted-proxy policy because request headers can be spoofed by untrusted clients.

Q8. What is a server block?

A server block defines request-handling rules for a listening address and host configuration, including server names, locations, and upstream behavior.

Q9. What is the difference between `root` and `alias`?

`root` combines the configured root with the request URI. `alias` maps a matched location to a specified filesystem path by replacing the matched location prefix.

Q10. How do you validate an Nginx configuration?

Run `nginx -t` with the appropriate configuration path. Inspect the complete active configuration with `nginx -T` when needed.

Q11. Why might a correct-looking `proxy_pass` produce a backend `404`?

The URI may be rewritten differently than expected, the wrong location may match, or the backend route may not exist. Compare the incoming URI with the URI forwarded upstream.

Q12. What is the difference between a reverse proxy and an API gateway?

A reverse proxy focuses on forwarding and traffic management. An API gateway may additionally provide API-oriented policies such as authentication integration, quotas, transformations, and centralized API lifecycle controls. Products can overlap, so compare actual capabilities rather than relying only on labels.

Q13. Why is Nginx often used for static files?

It can serve files directly without consuming application-server resources for every static asset request.

Q14. Why should you avoid blindly increasing proxy timeouts?

Longer timeouts can tie up connections and resources while an unhealthy backend remains slow. First determine whether the problem is legitimate workload, saturation, dependency latency, or a connectivity issue.

Q15. Does Nginx automatically make a backend secure?

No. It can contribute to security through TLS, routing, and request controls, but the system still needs secure application logic, authentication, authorization, network controls, secret management, and appropriate trust boundaries.

17. Production Scenario: API Returns `404` Only Through Nginx

Scenario: Calling the Spring Boot service directly works, but calling the public API through Nginx returns `404`.

Investigate in this order:

1. Reproduce the same HTTP method and URI against the direct backend.
2. Inspect the active `server_name` and determine which server block handles the request.
3. Identify the matching `location`.
4. Inspect the `proxy_pass` directive, especially whether it includes a trailing slash or URI component.
5. Compare the original request URI with the upstream URI.
6. Check Nginx access/error logs and application logs at the same timestamp.
7. Change only the incorrect routing or path-mapping rule and retest.

Key lesson: If direct backend access works but proxy access fails, the proxy's routing or URI transformation becomes a high-value investigation target.

18. Practical Mastery Assessment

Review your understanding of key Nginx concepts:

1. **Which component normally owns business rules?**
- *Options:* Nginx, Spring Boot, DNS
- *Correct Answer:* Spring Boot
- *Why:* Nginx manages traffic; the application implements business behavior.

2. **What should you run before reloading an edited Nginx configuration?**
- *Options:* `nginx -t`, `git reset --hard`, `redis-cli FLUSHALL`
- *Correct Answer:* `nginx -t`
- *Why:* It checks configuration syntax and related validity before applying changes.

3. **Nginx cannot connect to a stopped upstream. Which response is common?**
- *Options:* `502 Bad Gateway`, `201 Created`, `301 Moved Permanently`
- *Correct Answer:* `502 Bad Gateway`
- *Why:* An unavailable upstream connection commonly produces a `502`.

4. **What does `127.0.0.1` mean inside an Nginx container?**
- *Options:* The host in all cases, The Nginx container itself, The public DNS server
- *Correct Answer:* The Nginx container itself
- *Why:* Loopback refers to the current network namespace, not automatically another container.

5. Which location is an exact match for `/health`?

- *Options:* `location / { }`, `location = /health { }`, `location ~* .css { }`
- *Correct Answer:* `location = /health { }`
- *Why:* The equals modifier selects an exact URI match.

19. Hands-On Completion Checklist

- [ ] Explain forward proxy, reverse proxy, and load balancing without memorized definitions.
- [ ] Draw the request path from client to Nginx to Spring Boot.
- [ ] Identify the main Nginx configuration blocks and inspect the active configuration.
- [ ] Configure a server block with an exact-match health route and an API prefix.
- [ ] Reverse-proxy a local backend and verify the request with curl.
- [ ] Explain how `proxy_pass` trailing-slash behavior can change an upstream URI.
- [ ] Diagnose a deliberately stopped upstream using curl and the error log.
- [ ] Explain why localhost behaves differently inside a container.
- [ ] Describe the trust and security implications of forwarded headers.

20. Chunk 1 Summary

You should now understand Nginx as a traffic-management layer that can serve static files, select routes, and proxy requests to backend applications. You have practiced the configuration hierarchy, server blocks, location matching, `proxy_pass`, forwarded headers, and a structured debugging workflow.

Next: Nginx — Chunk 2 of 2: Production Load Balancing, TLS, Reliability, Security & Incident Mastery.

That final chunk will complete the Nginx curriculum and cover production-grade upstream configuration, health and failure behavior, TLS termination, operational observability, performance tradeoffs, incident simulations, and advanced interview preparation.

PART II: PRODUCTION LOAD BALANCING, TLS, RELIABILITY, SECURITY & INCIDENT MASTERY

Chunk 2 of 2 · Backend Engineering Handbook

Nginx — Chunk 2 of 2: Production Load Balancing, TLS, Reliability, Security & Incident Mastery

Backend Engineering Tools & Infrastructure Handbook

Source file: `content/nginx/02-nginx-production-load-balancing-tls-reliability-security.md`

Learning Objectives

By the end of this chapter, you should be able to:

- Configure Nginx to distribute requests across multiple backend instances.
- Explain upstream health, failure handling, connection limits, and timeouts.
- Understand TLS termination, HTTPS redirection, and certificate renewal.
- Configure production-minded request limits, headers, and access controls.
- Diagnose latency, connection failures, unhealthy upstreams, and deployment regressions.
- Understand Nginx logs, metrics, and the limits of proxy-level observability.
- Make informed tradeoffs about caching, buffering, and connection management.
- Answer production-focused Nginx interview questions.

This completes the two-chunk Nginx curriculum. The next stage is to save and verify both source files before producing the final Nginx handbook PDF.

Layer 1 — Fundamentals & Mental Models

1. From a Single Backend to a Production Traffic Layer

A single-instance deployment might look like this:

```
Clients
   |
   v
 Nginx
   |
   v
Spring Boot :8080
   |
   v
PostgreSQL
```

This is simple, but the application is a single point of failure. If Spring Boot stops, Nginx cannot successfully serve dynamic API requests.

A horizontally scaled design introduces multiple application instances:

```
                    +--> Spring Boot :8081
Clients --> Nginx --+--> Spring Boot :8082
                    +--> Spring Boot :8083
                              |
                              v
                         PostgreSQL
```

Nginx can distribute incoming requests among configured upstream servers.

However, multiple instances introduce new questions:

- How are requests distributed?
- What happens when one instance fails?
- How do we avoid sending traffic to a starting or unhealthy instance?
- How are application versions deployed safely?
- How do we identify whether latency comes from Nginx, the application, or a downstream dependency?

Production Nginx configuration is about answering these questions deliberately, not merely adding more servers.

2. Upstream Groups and Load-Balancing Algorithms

An `upstream` block defines a group of backend servers.

```
http {
    upstream orders_api {
        server 127.0.0.1:8081;
        server 127.0.0.1:8082;
        server 127.0.0.1:8083;
    }

    server {
        listen 80;
        server_name api.example.com;

        location / {
            proxy_pass http://orders_api;
        }
    }
}
```

By default, Nginx distributes requests using a round-robin strategy.

Common strategies

Strategy	Mental model	Useful when
Round robin	Distribute requests in rotation	Backends have broadly similar capacity
Weighted round robin	Give some servers a larger share	Instances have different capacities
Least connections	Prefer a server with fewer active connections	Request durations vary significantly
IP hash	Use client IP information to influence server selection	A limited form of affinity is needed

The available methods and behavior depend on the Nginx edition, module, and configuration.

Weighted round robin

```
upstream orders_api {
    server 127.0.0.1:8081 weight=3;
    server 127.0.0.1:8082 weight=1;
}
```

This gives the first server a larger share of traffic over time. It does not guarantee an exact percentage over every short observation window.

Least connections

```
upstream orders_api {
    least_conn;

    server 127.0.0.1:8081;
    server 127.0.0.1:8082;
    server 127.0.0.1:8083;
}
```

This can be useful when some requests remain active much longer than others. But active connection count is only a proxy for workload: it does not directly measure CPU saturation, database contention, or request complexity.

Interview takeaway: A load-balancing algorithm distributes traffic; it does not automatically make a backend horizontally scalable. The application and its dependencies must tolerate concurrent requests and shared state.

3. Health Checks and Failure Handling

A backend can fail in several ways:

- Its process is stopped.
- Its port is not listening.
- It accepts connections but responds too slowly.
- It returns application errors.
- It is alive but cannot reach PostgreSQL or another critical dependency.

These conditions are not equivalent.

Passive failure detection

In common Nginx Open Source configurations, upstream failure handling is primarily passive: Nginx observes failures while attempting real requests and can temporarily mark an upstream server unavailable according to configured parameters.

```
upstream orders_api {
    server 127.0.0.1:8081 max_fails=3 fail_timeout=10s;
    server 127.0.0.1:8082 max_fails=3 fail_timeout=10s;
}
```

Here, `max_fails` and `fail_timeout` influence when an upstream server is considered temporarily unavailable under the applicable conditions.

These settings are not a universal active-health-check system. Exact behavior depends on the upstream configuration and Nginx edition.

Active health checks

Active health checks periodically probe a backend independently of normal user traffic. These capabilities depend on the Nginx edition or an additional health-check mechanism.

A Spring Boot health endpoint might be:

```
/actuator/health
```

The endpoint must be enabled and safely exposed. A basic process-health check is not necessarily proof that every dependency or business operation is healthy.

Health-check design

A useful health strategy distinguishes:

- **Liveness:** is the application process functioning?
- **Readiness:** should this instance receive new traffic?
- **Dependency health:** can it access the resources needed to serve requests?

Avoid turning every transient downstream issue into an automatic restart loop. Health semantics should reflect the recovery behavior of the application and its platform.

4. Timeouts: Three Different Clocks

A proxy timeout is not the same as an application execution deadline.

Three important Nginx directives are:

```
proxy_connect_timeout 3s;
proxy_send_timeout    30s;
proxy_read_timeout    30s;
```

- `proxy_connect_timeout`: time allowed to establish an upstream connection.
- `proxy_send_timeout`: timeout between successive writes while sending a request to the upstream.
- `proxy_read_timeout`: timeout between successive reads from the upstream.

A critical nuance: `proxy_read_timeout` generally measures inactivity between read operations, not the total wall-clock duration of the entire response.

For example, a streaming response that sends data periodically can remain active longer than the configured read timeout, while a backend that sends nothing for too long can trigger a timeout.

Why blindly increasing timeouts is dangerous

Suppose an endpoint is slow because it performs a poorly indexed database query.

Increasing the proxy timeout might stop clients from receiving errors immediately, but it may also keep connections occupied longer and increase pressure during overload.

A better sequence is:

1. Measure request latency.
2. Check application and database traces or logs.
3. Identify whether the delay is expected or pathological.
4. Set timeouts consistent with the API's latency budget.
5. Apply concurrency controls and overload protection where appropriate.

5. TLS Termination and HTTPS

TLS protects data in transit between two endpoints.

A common deployment terminates public HTTPS at Nginx:

```
Client
  |
  | HTTPS
  v
Nginx :443
  |
  | HTTP on a restricted internal network
  v
Spring Boot :8080
```

TLS termination lets Nginx handle the public certificate and encrypted client connection.

Whether the internal hop should also use TLS depends on the network trust boundary and security requirements. In untrusted or regulated environments, encrypting internal traffic may be necessary.

Example HTTPS server block

```
server {
    listen 443 ssl;
    server_name api.example.com;

    ssl_certificate     /etc/letsencrypt/live/api.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.example.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:8080;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

The certificate paths are examples. They must exist and contain a valid certificate and private key for the deployed hostname.

Redirect HTTP to HTTPS

```
server {
    listen 80;
    server_name api.example.com;

    return 301 https://$host$request_uri;
}
```

This sends clients to the HTTPS URL while preserving the requested path and query string.

Before enabling a redirect, ensure HTTPS is configured and working. Otherwise, clients may be redirected to an unusable endpoint.

Certificate operations

A production certificate lifecycle includes:

1. Issuing the certificate for the correct hostname.
2. Protecting the private key.
3. Installing the certificate chain and key.
4. Testing the Nginx configuration.
5. Reloading Nginx after renewal.
6. Monitoring expiration and renewal failures.

A certificate that was valid when first installed can still expire later. Renewal monitoring is therefore part of operations, not merely setup.

6. Connection Reuse and Upstream Keepalive

Repeatedly establishing upstream connections can add overhead.

Nginx can reuse upstream connections when the configuration, HTTP protocol, and upstream application support it.

Example pattern:

```
upstream orders_api {
    server 127.0.0.1:8081;
    server 127.0.0.1:8082;

    keepalive 32;
}

server {
    listen 80;
    server_name api.example.com;

    location / {
        proxy_http_version 1.1;
        proxy_set_header Connection "";

        proxy_pass http://orders_api;
    }
}
```

This configures a per-worker cache of idle upstream keepalive connections. It does **not** impose a global maximum of 32 connections across all workers, nor does it mean every request will reuse a connection.

Keepalive can reduce connection-establishment overhead, but it must be configured in the context of backend connection limits, proxy concurrency, and the actual traffic pattern.

7. Buffering and Streaming

Nginx can buffer proxied responses and requests.

Response buffering

With response buffering enabled, Nginx can read data from an upstream and buffer it before delivering all of it to the client. This can help protect the application from slow clients and decouple upstream response progress from downstream consumption.

However, large responses can consume memory or use temporary files.

Streaming

For streaming APIs, server-sent events, or other responses that must reach clients incrementally, buffering behavior needs deliberate configuration.

Example:

```
location /events/ {
    proxy_pass http://events_api;
    proxy_buffering off;
}
```

Whether this is appropriate depends on the application protocol. Disabling buffering indiscriminately can reduce efficiency or expose upstreams to slow-client pressure.

Rule: Choose buffering settings based on measured behavior and protocol requirements, not by copying a generic configuration.

Layer 2 — Practical Engineering & Workflows

8. Build a Local Load-Balancing Lab

This lab demonstrates request distribution without requiring paid cloud infrastructure.

Step 1 — Start two backend instances

If you have a Spring Boot application that supports a configurable port, start two instances using different ports. For example:

```
SERVER_PORT=8081 java -jar target/app.jar
```

In another terminal:

```
SERVER_PORT=8082 java -jar target/app.jar
```

These commands assume the application honors `SERVER_PORT` and that the built JAR exists. Use your project's actual JAR path and configuration.

For an observable test, expose a harmless diagnostic endpoint that identifies the instance. Do not expose sensitive environment information.

Step 2 — Configure an upstream group

```
upstream demo_api {
    server 127.0.0.1:8081;
    server 127.0.0.1:8082;
}

server {
    listen 8088;
    server_name localhost;

    location / {
        proxy_pass http://demo_api;

        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Ensure this configuration is included in the active Nginx `http` context.

Step 3 — Test and reload

```
nginx -t
```

Then reload Nginx using the appropriate service command for your installation.

Step 4 — Send repeated requests

```
for i in $(seq 1 10); do
    curl -s http://127.0.0.1:8088/instance
    echo
done
```

Replace `/instance` with your diagnostic endpoint.

Observe which instances handle the requests. Do not expect exact alternation in every setup if other configuration, connection behavior, or traffic is involved.

Step 5 — Stop one backend

Stop the instance on port `8081` and repeat the test.

Observe the failure behavior and inspect the error log. Passive failure handling may temporarily avoid an upstream after enough qualifying failures, but the behavior depends on the configured failure conditions.

Step 6 — Restore and document

Restart the stopped backend, repeat the test, and record:

- Whether both instances received traffic.
- How quickly a failed instance was avoided, if applicable.
- What errors appeared during failure.
- Whether recovery was automatic.
- What would need to change for explicit readiness-based health checks.

This lab teaches more than syntax: it demonstrates why failover behavior must be tested rather than assumed.

9. Production-Minded Configuration Example

The following example combines upstream routing, request limits, and conservative proxy timeouts. Adapt it to your environment rather than treating it as a universal production template.

```
upstream orders_api {
    least_conn;

    server 127.0.0.1:8081 max_fails=3 fail_timeout=10s;
    server 127.0.0.1:8082 max_fails=3 fail_timeout=10s;

    keepalive 32;
}

server {
    listen 443 ssl;
    server_name api.example.com;

    ssl_certificate     /path/to/fullchain.pem;
    ssl_certificate_key /path/to/privkey.pem;

    client_max_body_size 10m;

    location /api/ {
        proxy_pass http://orders_api;

        proxy_http_version 1.1;
        proxy_set_header Connection "";

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        proxy_connect_timeout 3s;
        proxy_send_timeout 30s;
        proxy_read_timeout 30s;
    }
}
```

Before deploying it, verify:

- The certificate and key exist.
- The upstream instances are reachable from Nginx.
- The URI behavior matches the API's route design.
- The body-size limit fits legitimate uploads.
- The timeout values fit the API's latency budget.
- Forwarded headers are trusted only through the intended proxy chain.
- Logs, monitoring, and certificate renewal are configured.
- The configuration is compatible with the installed Nginx version.

10. Access Controls and Request Limits

Restrict access to an internal endpoint

```
location /internal/ {
    allow 10.0.0.0/8;
    deny all;

    proxy_pass http://internal_api;
}
```

This example permits the specified private address range and denies other addresses based on the client address Nginx sees.

If Nginx is behind another proxy or load balancer, that address may be the upstream proxy rather than the original client. Configure real-client-IP handling only with trusted proxy addresses and a deliberate trust policy.

IP-based restrictions are not a replacement for authentication and authorization.

Request-body limits

```
client_max_body_size 10m;
```

A limit that is too small may reject legitimate requests with `413 Content Too Large`. A limit that is too large can allow unnecessarily expensive uploads.

Choose limits based on the API contract and application constraints.

Rate limiting

Nginx can apply request-rate controls using its request-limiting modules.

```
http {
    limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;

    server {
        listen 8088;

        location /api/ {
            limit_req zone=api_limit burst=20 nodelay;
            proxy_pass http://127.0.0.1:8080;
        }
    }
}
```

This is a basic example of per-client-address rate limiting. Its behavior depends on the configured zone, burst handling, and client identity.

Important production considerations:

- A single client IP may represent many users behind a NAT.
- An untrusted forwarding header should not be used as an identity key.
- Multiple Nginx instances do not automatically share local rate-limit counters.
- Distributed rate limits may require application-level or shared-state coordination.
- Legitimate bursts and retry storms should be considered.

11. Observability: Logs, Metrics, and Traces

Nginx provides valuable evidence about traffic, but it does not explain every application failure by itself.

Access logs

Access logs can capture fields such as:

- Request method and URI.
- HTTP status code.
- Response size.
- Request processing time.
- Upstream response time.
- Upstream address.

A custom log format can help distinguish proxy overhead from upstream latency:

```
http {
    log_format upstream_timing
        '$remote_addr "$request" status=$status '
        'request_time=$request_time '
        'upstream_time=$upstream_response_time '
        'upstream_addr=$upstream_addr';

    access_log /var/log/nginx/access.log upstream_timing;
}
```

Field availability and values depend on request flow and whether an upstream was contacted.

How to interpret the timings

- **`request_time`** describes elapsed time processing the request from Nginx's perspective.
- **`upstream_response_time`** records upstream response timing information.
- A high upstream time suggests investigating the backend or its dependencies.
- A low upstream time with a slow overall request may indicate downstream transfer delays, buffering behavior, or other proxy/client-side factors.

These are clues, not conclusive root-cause proof. Correlate them with application metrics, database telemetry, and distributed traces.

Metrics

Production monitoring may include:

- Request rate.
- Status-code distribution.
- Request latency percentiles.
- Active connections.
- Upstream failures.
- Backend saturation.
- TLS and certificate status.

Some metrics require an Nginx status endpoint, an exporter, or other instrumentation. Protect status endpoints so they do not unintentionally reveal operational details to the public.

Tracing

Nginx access logs and metrics can help locate a slow hop, while distributed tracing can follow a request through the application and its dependencies.

Use a consistent request or trace identifier across the proxy and backend where supported. Do not trust arbitrary client-supplied identifiers for security decisions.

12. Graceful Reloads and Deployment Safety

A configuration change should not be deployed blindly.

Typical workflow:

```
nginx -t &&
sudo systemctl reload nginx
```

On systems using systemd, the reload command asks the service to apply configuration changes without treating it as a full stop-and-start operation. Exact behavior depends on the package and service configuration.

A safer deployment sequence is:

1. Validate configuration syntax.
2. Review the diff.
3. Confirm referenced files, certificates, and upstreams.
4. Reload the service.
5. Run a synthetic request through the public route.
6. Inspect status codes, latency, and logs.
7. Roll back the change if the regression is confirmed.

A reload command returning successfully does not establish that the new routing works for every hostname or API path.

Layer 3 — Production Engineering & Interview Mastery

13. Incident Simulation A — All API Requests Return `502`

Symptoms: The Nginx endpoint is reachable, but requests to the API return `502 Bad Gateway`.

Investigation

1. Confirm the response is generated by Nginx rather than the application.
2. Check whether the backend process is running.
3. Verify the actual listening address and port.
4. Test the upstream directly from the Nginx host or container.
5. Inspect Nginx error logs for connection refusal, resolution failures, or other upstream errors.
6. Verify container networking and whether `localhost` points to the intended service.
7. Restore connectivity and retest the public endpoint.

Example commands

```
curl -i http://127.0.0.1:8081/actuator/health
sudo ss -ltnp
sudo tail -n 100 /var/log/nginx/error.log
```

Adapt commands to the environment and the application's enabled endpoints.

Common root causes: backend not started, incorrect port, incorrect container hostname, network restrictions, or an upstream service that is unavailable.

14. Incident Simulation B — Latency Spikes, but Nginx Is Healthy

Symptoms: The proxy responds, but API latency increases dramatically.

Investigation

1. Compare overall request time with upstream response time.
2. Check Spring Boot request latency, thread pools, CPU, and memory.
3. Inspect database query latency and connection-pool saturation.
4. Check Redis, Kafka, and other dependencies if the affected request relies on them.
5. Compare latency by endpoint, status code, and time window.
6. Determine whether the incident affects one backend instance or all instances.
7. Apply a targeted mitigation and confirm recovery through measured latency.

Do not assume Nginx is responsible merely because the error appears at the public endpoint. The proxy may simply be revealing a slow upstream.

15. Incident Simulation C — One Backend Instance Is Unhealthy

Symptoms: Some requests fail while others succeed.

Potential causes include:

- One application instance is down.
- One instance has a configuration mismatch.
- One instance cannot reach the database.
- One instance is overloaded.
- The load-balancing policy is not behaving as expected.

Response

1. Identify which upstream address correlates with failures.
2. Test each backend directly.
3. Compare application versions and configuration.
4. Review upstream failure logs and configured passive-failure parameters.
5. Remove or isolate the unhealthy instance using the approved operational process.
6. Verify that remaining capacity can handle the traffic.

7. Restore the instance only after it passes the appropriate readiness checks.

Avoid repeatedly reintroducing an unhealthy instance without understanding why it fails.

16. Incident Simulation D — HTTPS Works, but Redirected Requests Fail

Symptoms: HTTP requests redirect to HTTPS, but the destination returns a certificate warning or an error.

Investigate:

1. Does the HTTPS listener exist?
2. Is the certificate valid for the requested hostname?
3. Is the certificate chain configured correctly?
4. Does the private key match the certificate?
5. Is the certificate expired or not yet valid?
6. Is Nginx serving the intended virtual host?
7. Is the HTTPS upstream routing correct after TLS terminates?
8. Does the application incorrectly generate HTTP redirects because it does not correctly understand trusted forwarded-protocol information?

Fix the underlying certificate or proxy/application configuration. Disabling certificate verification is not a sound production fix.

17. Incident Simulation E — Requests Fail After an Nginx Configuration Change

Symptoms: An API route previously worked but now returns `404`, `403`, `413`, or `502`.

Response checklist

- Compare the deployed configuration with the last known-good version.
- Run `nginx -T` and identify the active server block and location.
- Check whether the `proxy_pass` URI changed.
- Verify upstream hostnames and ports.
- Check request-size and access-control directives.
- Validate the configuration with `nginx -t`.
- Roll back the specific regression if needed.
- Re-run smoke tests for critical routes.
- Record the root cause and add a regression test or deployment check.

The fastest recovery is often a controlled rollback, not a sequence of unverified configuration edits.

18. Reliability and Capacity Tradeoffs

One Nginx instance is still a single point of failure

Multiple application instances behind one Nginx process do not remove the proxy itself as a failure point.

Production systems may place multiple Nginx instances behind a cloud load balancer, virtual IP, or another supported entry-point mechanism.

Proxy capacity is not backend capacity

Nginx may handle many open network connections, while the backend is limited by CPU, thread pools, database connections, or other dependencies.

Monitor both layers.

Retries can amplify incidents

Retrying failed requests may improve resilience for certain transient failures, but retries can also amplify load during an outage.

Consider:

- Which methods and operations are safe to retry.
- Whether a request may have completed before a connection failed.
- Idempotency keys and duplicate side effects.
- Retry limits and backoff.
- The combined retry behavior of clients, proxies, and application code.

Never assume that a failed response means the backend performed no work.

Sticky sessions have costs

Client affinity may help legacy stateful applications, but it can reduce flexibility in distributing load and complicate failover. Prefer shared or externalized session state when appropriate, while considering the added dependency and its failure modes.

19. Advanced Interview Questions

Q1. How does Nginx choose an upstream server?

It uses the configured upstream group and load-balancing method. Round robin is the default for a standard upstream group; methods such as least connections and IP hash can be configured when suitable.

Q2. What is the difference between passive and active health checks?

Passive checks infer failure from real traffic and upstream errors. Active checks independently probe backend health at intervals. Active-check support depends on the edition or additional modules used.

Q3. Does `max_fails` guarantee that Nginx will detect every unhealthy backend?

No. It influences passive failure handling under the applicable conditions. It is not a complete application-readiness check and does not guarantee detection of every logical failure.

Q4. What is the difference between `proxy_connect_timeout` and `proxy_read_timeout`?

The connection timeout controls establishing the upstream connection. The read timeout controls the allowed inactivity between upstream read operations.

Q5. Why can a backend return `404` only when accessed through Nginx?

The proxy may select the wrong location, alter the URI through `proxy_pass`, or route to a different upstream. Compare direct and proxied requests and inspect the active configuration.

Q6. What is TLS termination?

It is the point at which an endpoint accepts the encrypted TLS connection and handles decryption. Nginx can terminate client TLS and forward the request to an internal HTTP or HTTPS upstream.

Q7. Is internal HTTP always safe after TLS termination?

No. Its suitability depends on the network trust boundary, infrastructure controls, and compliance requirements. Internal TLS may still be required.

Q8. What is the purpose of upstream keepalive?

It can reuse idle upstream connections and reduce connection-establishment overhead. It must be configured with the relevant HTTP and upstream behavior in mind.

Q9. What do `request_time` and `upstream_response_time` tell you?

They provide different views of request and upstream timing from Nginx's perspective. Comparing them helps narrow down where latency may be introduced, but neither replaces application-level telemetry.

Q10. How would you protect an internal API route?

Use appropriate network restrictions, authentication and authorization, and careful proxy trust configuration. IP allowlists can be useful but should not be the only security mechanism.

Q11. Why can a timeout increase worsen an outage?

Longer timeouts may keep more requests and connections occupied during backend overload, increasing resource pressure and delaying failure detection.

Q12. How do retries interact with idempotency?

A retry may repeat an operation that already completed despite the client not receiving its response. Non-idempotent operations require safeguards such as idempotency keys or deduplication to prevent duplicate effects.

Q13. Does round robin guarantee equal CPU usage across instances?

No. It distributes requests according to the configured policy, not according to actual CPU cost. Requests may have different execution times and resource requirements.

Q14. How would you investigate intermittent upstream errors?

Correlate errors by upstream address, request time, status code, and application logs. Test instances individually and look for differences in configuration, resource saturation, network behavior, and dependency health.

Q15. What is the relationship between Nginx and Kubernetes Ingress?

Nginx can be used as an ingress controller or as a proxy outside Kubernetes. Kubernetes Ingress is an API abstraction; an ingress controller implements the routing behavior. The specific Nginx integration and its capabilities depend on the controller and version.

Q16. Why should status endpoints be protected?

They may reveal traffic levels, connection counts, and internal operational details. Expose only the information needed and restrict access appropriately.

Q17. When is caching at Nginx appropriate?

It can help when responses are safely cacheable and invalidation semantics are understood. Personalized, authorization-sensitive, or rapidly changing responses need careful cache-key and policy design.

Q18. What would you monitor for a public API proxy?

Request rate, status-code distribution, latency percentiles, upstream errors, active connections, backend saturation, certificate expiration, and relevant application/dependency metrics.

Q19. Does multiple upstream configuration automatically provide high availability?

No. It helps distribute requests and can tolerate some instance failures, but only if failure behavior, remaining capacity, shared dependencies, network paths, and the proxy entry point itself are addressed.

Q20. How do you deploy an Nginx change safely?

Review the diff, test configuration, verify referenced resources, reload using the appropriate service mechanism, run synthetic smoke tests, inspect logs and latency, and roll back if necessary.

20. Capstone: Put a Spring Boot API Behind Nginx

Build a local setup with:

- One Nginx reverse proxy.
- Two Spring Boot instances.
- A PostgreSQL dependency.
- A health endpoint suitable for operational checks.
- Access logs containing upstream timing.
- A small smoke-test script.
- A documented recovery procedure.

Required exercises

1. Route requests to both backend instances.
2. Confirm the upstream URI matches the controller mapping.
3. Stop one backend and observe failure behavior.
4. Restore the instance and verify recovery.
5. Introduce a controlled slow endpoint and compare request timing with upstream timing.
6. Configure a local HTTPS test setup if you have suitable development certificates.
7. Apply a request-size limit and verify the expected response to an oversized request.
8. Document how you would secure an internal endpoint.
9. Record the steps required to roll back a faulty proxy configuration.
10. Explain which problems belong to Nginx, Spring Boot, PostgreSQL, or the deployment network.

Do not expose diagnostic endpoints or development credentials publicly. Use synthetic data for the lab.

21. Final Nginx Mastery Checklist

- [ ] I can explain reverse proxying and load balancing.
- [ ] I can write and validate a basic upstream configuration.
- [ ] I understand round robin, weighted distribution, least connections, and IP hash.
- [ ] I understand passive failure handling and its limitations.
- [ ] I can distinguish connection, send, and read timeouts.
- [ ] I can explain TLS termination and certificate renewal.
- [ ] I understand the role of upstream keepalive.
- [ ] I can reason about buffering and streaming.
- [ ] I can apply request-size and rate limits appropriately.
- [ ] I can interpret Nginx access logs and upstream timing.
- [ ] I can investigate `502`, `504`, routing, TLS, and configuration incidents.
- [ ] I understand why proxy health does not prove application health.
- [ ] I can describe a safe configuration deployment and rollback.
- [ ] I can explain the tradeoffs of retries, affinity, and caching.

22. Nginx Curriculum Completion Protocol

Nginx content is complete across its two planned chunks.

Expected authoritative Markdown files:

```
content/nginx/
■■■ 01-nginx-reverse-proxies-routing-core-mental-models.md
■■■ 02-nginx-production-load-balancing-tls-reliability-security.md
```

After both files are saved and verified:

1. Review both chapters for continuity, examples, and technical consistency.
2. Confirm all three learning layers are covered across the two chunks.
3. Generate the **single final Nginx handbook PDF**, not PDFs per chunk.
4. Save it as:

```
docs/nginx/BEH_NGINX_COMPLETE_HANDBOOK.pdf
```

5. Verify the PDF includes both complete chapters, code blocks, tables, and interview questions.
6. Update the README using verified completion status.
7. Commit the completed Nginx source and final PDF with the exact commit message:

```
Nginx done
```

Do not mark the source, PDF, or Git commit complete until the corresponding work has actually been saved and verified.

Nginx is now complete at the curriculum-content level. The next tool in the locked sequence is **Prometheus — Chunk 1 of 2: Metrics, Time Series & Monitoring Fundamentals**.